03. dstack TDX Quote 생성 및 검증 상세 분석

목차

1. TDX Quote 시스템 개요
2. Quote 생성 과정
3. Quote 구조 분석
4. Quote 검증 과정
5. Event Log 재생
6. 애플리케이션 정보 추출
7. TCB 검증
8. 인증서 통합

---

1. TDX Quote 시스템 개요

1.1 Quote의 역할

TDX Quote는 TEE 환경에서 실행 중인 애플리케이션이 **신뢰할 수 있는 상태**임을 외부에 증명하는 핵심 메커니즘입니다.

1.2 핵심 구성 요소

```
// ra-tls/src/attestation.rs

/// Attestation data
```

```rust
#[derive(Debug, Clone)]
pub struct Attestation<R = ()> {
/// Quote - TDX 하드웨어에서 생성된 증명 데이터
pub quote: Vec<u8>,
/// Raw event log - 원시 이벤트 로그 데이터
pub raw_event_log: Vec<u8>,
/// Event log - 파싱된 이벤트 로그
pub event_log: Vec<EventLog>,
/// Verified report - 검증된 리포트
pub report: R,
}

/// Represents a verified attestation
pub type VerifiedAttestation = Attestation<VerifiedReport>;
```

1.3 Quote 콘텐츠 타입

```rust
// ra-tls/src/attestation.rs
/// The content type of a quote. A CVM should only generate quotes for these typ
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum QuoteContentType<'a> {
/// The public key of KMS root CA
KmsRootCa,
```

```
/// The public key of the RA-TLS certificate
RaTlsCert,
/// App defined data
AppData,
/// The custom content type
Custom(&'a str),
}


impl QuoteContentType<'_> {
/// The tag of the content type used in the report data.
pub fn tag(&self) -> &str {
match self {
Self::KmsRootCa => "kms-root-ca",
Self::RaTlsCert => "ratls-cert",
Self::AppData => "app-data",
Self::Custom(tag) => tag,
}
}
}
```

---

2. Quote 생성 과정

2.1 Guest Agent에서의 Quote 요청

```
// guest-agent/src/rpc_service.rs

impl DstackGuestRpc for InternalRpcHandler {

async fn get_quote(self, request: RawQuoteArgs) -> Result<GetQuoteResponse> {

// 1. Report Data 패딩 (64바이트로 맞춤)

fn pad64(data: &[u8]) -> Option<[u8; 64]> {

if data.len() > 64 {

return None;

}

let mut padded = [0u8; 64];

padded[..data.len()].copy_from_slice(data);

Some(padded)

}

let report_data = pad64(&request.report_data)

.context("Report data is too long")?;

// 2. 시뮬레이터 모드 확인

if self.state.config().simulator.enabled {

return simulate_quote(

self.state.config(),

report_data,

&self.state.inner.vm_config,

);
```

```
}

// 3. 실제 TDX Quote 요청

let (_, quote) = tdx_attest::get_quote(&report_data, None)

.context("Failed to get quote")?;

// 4. Event Log 읽기

let  event_log = read_event_logs().context("Failed to decode event log")?;

let  event_log = serde_json::to_string(&event_log)

.context("Failed to serialize event log")?;



Ok(GetQuoteResponse {

quote,

event_log,

report_data: report_data.to_vec(),

vm_config: self.state.inner.vm_config.clone(),

})

}

}
```

2.2 시스템 레벨 Quote 생성

```
// tdx-attest/src/linux.rs

pub  fn  get_quote(

report_data: &TdxReportData,
```

```
att_key_id_list: Option<&[TdxUuid]>,
) -> Result<(TdxUuid, Vec<u8>)> {
let mut att_key_id = TdxUuid([0; TDX_UUID_SIZE as usize]);
let mut quote_ptr = ptr::null_mut();
let mut quote_size = 0;

// 1. TDX 시스템 호출 실행
let error = unsafe {
let key_id_list_ptr = att_key_id_list
.map(|list| list.as_ptr() as *const tdx_uuid_t)
.unwrap_or(ptr::null());
tdx_att_get_quote(
report_data as *const TdxReportData as *const tdx_report_data_t,
key_id_list_ptr,
att_key_id_list.map_or(0, |list| list.len() as u32),
&mut att_key_id as *mut TdxUuid as *mut tdx_uuid_t,
&mut quote_ptr,
&mut quote_size,
0,
)
};

// 2. 오류 확인
```

```
if error != _tdx_attest_error_t::TDX_ATTEST_SUCCESS {
return Err(error.into());
}

// 3. Quote 데이터 복사
let quote = unsafe {
slice::from_raw_parts(quote_ptr, quote_size as usize).to_vec()
};

// 4. 메모리 해제
unsafe {
tdx_att_free_quote(quote_ptr);
}

Ok((att_key_id, quote))
}
```

2.3 Report Data 생성

```
// ra-tls/src/attestation.rs
impl QuoteContentType<'_> {
/// Convert the content to the report data with a specific hash algorithm.
pub fn to_report_data_with_hash(&self, content: &[u8], hash: &str) -> Result<[
```

```
macro_rules!  do_hash {
($hash: ty) => {{
// The format is: hash(<tag>:<content>)
let  mut  hasher = <$hash>::new();
hasher.update(self.tag().as_bytes());
hasher.update(b":");
hasher.update(content);
let  output = hasher.finalize();


let  mut  padded = [0u8; 64];
padded[..output.len()].copy_from_slice(&output);
padded
}};
}
let  hash = if  hash.is_empty() {
DEFAULT_HASH_ALGORITHM // "sha512"
} else {
hash
};
let  output = match  hash {
"sha256" => do_hash!(sha2::Sha256),
"sha384" => do_hash!(sha2::Sha384),
"sha512" => do_hash!(sha2::Sha512),
```

```
"sha3-256" => do_hash!(sha3::Sha3_256),

"sha3-384" => do_hash!(sha3::Sha3_384),

"sha3-512" => do_hash!(sha3::Sha3_512),

"keccak256" => do_hash!(sha3::Keccak256),

"keccak384" => do_hash!(sha3::Keccak384),

"keccak512" => do_hash!(sha3::Keccak512),

"raw" => content.try_into().ok().context("invalid content length")?,

_ => bail!("invalid hash algorithm"),

};

Ok(output)

}



/// Convert the content to the report data.

pub  fn  to_report_data(&self, content: &[u8]) -> [u8; 64] {

self.to_report_data_with_hash(content, "")

.expect("sha512 hash should not fail")

}

}
```

---

3. Quote 구조 분석

3.1 Quote 파싱

```rust
// ra-tls/src/attestation.rs

impl<T> Attestation<T> {

/// Decode the quote

pub fn decode_quote(&self) -> Result<Quote> {

Quote::parse(&self.quote)

}



/// Decode the report data in the quote

pub fn decode_report_data(&self) -> Result<[u8; 64]> {

match self.decode_quote()?.report {

Report::SgxEnclave(report) => Ok(report.report_data),

Report::TD10(report) => Ok(report.report_data),

Report::TD15(report) => Ok(report.base.report_data),

}

}

}
```

3.2 TDX Report 구조

```rust
// qvl 라이브러리에서 정의된 구조체들

pub enum Report {

SgxEnclave(EnclaveReport),

TD10(TDReport10), // TDX 1.0 리포트
```

```
TD15(TDReport15), // TDX 1.5 리포트
}

// TDReport10 구조체 (주요 필드들)
pub struct TDReport10 {
pub mr_td: [u8; 48], // TD 측정값
pub mr_config_id: [u8; 48], // 설정 ID 측정값
pub mr_owner: [u8; 48], // 소유자 측정값
pub mr_owner_config: [u8; 48], // 소유자 설정 측정값
pub rt_mr0: [u8; 48], // RTMR0
pub rt_mr1: [u8; 48], // RTMR1
pub rt_mr2: [u8; 48], // RTMR2
pub rt_mr3: [u8; 48], // RTMR3
pub report_data: [u8; 64], // 리포트 데이터
pub td_attributes: [u8; 8], // TD 속성
// ... 기타 필드들
}
```

---

4. Quote 검증 과정

4.1 전체 검증 플로우

```rust
// ra-tls/src/attestation.rs

impl Attestation {

/// Verify the quote

pub  async  fn  verify(

self,

report_data: &[u8; 64],

pccs_url: Option<&str>,

) -> Result<VerifiedAttestation> {

let  quote = &self.quote;

// 1. Report Data 검증

if &self.decode_report_data()? != report_data {

bail!("report data mismatch");

}

// 2. PCCS URL 결정

let  mut  pccs_url = Cow::Borrowed(pccs_url.unwrap_or_default());

if  pccs_url.is_empty() {

// 환경 변수에서 PCCS_URL 읽기 시도

pccs_url = match std::env::var("PCCS_URL") {

Ok(url) => Cow::Owned(url),

Err(_) => Cow::Borrowed(""),

};

}
```

```
// 3. Quote 검증 (Intel DCAP QVL 사용)
let  report = qvl::collateral::get_collateral_and_verify(quote, Some(pccs_url.as
.await
.context("Failed to get collateral")?;
// 4. RTMR 검증 (TDX 리포트인 경우)
if  let Some(report) = report.report.as_td10() {
// Event Log 재생을 통한 RTMR 검증
let  rtmrs = self
.replay_event_logs(None)
.context("Failed to replay event logs")?;
if  rtmrs != [report.rt_mr0, report.rt_mr1, report.rt_mr2, report.rt_mr3] {
bail!("RTMR mismatch");
}
}
// 5. TCB 검증
validate_tcb(&report)?;
Ok(VerifiedAttestation {
quote: self.quote,
raw_event_log: self.raw_event_log,
event_log: self.event_log,
report,
})
}
```

```
}
```

4.2 RA-TLS 인증서 검증

```
// ra-tls/src/attestation.rs
impl Attestation {
/// Verify the quote with RA-TLS public key
pub  async  fn  verify_with_ra_pubkey(
self,
ra_pubkey_der: &[u8],
pccs_url: Option<&str>,
) -> Result<VerifiedAttestation> {
self.verify(
&QuoteContentType::RaTlsCert.to_report_data(ra_pubkey_der),
pccs_url,
)
.await
}
}
```

---

5. Event Log 재생

5.1 Event Log 구조

```rust
// cc-eventlog 라이브러리에서 정의
pub struct TdxEventLog {
pub imr: u32, // IMR 인덱스 (0-3)
pub event_type: u32, // 이벤트 타입
pub digest: [u8; 48], // SHA-384 다이제스트
pub event: String, // 이벤트 이름
pub event_payload: Vec<u8>, // 이벤트 페이로드
}
```

5.2 Event Log 재생 과정

```rust
// ra-tls/src/attestation.rs
/// Replay event logs
pub fn replay_event_logs(eventlog: &[EventLog], to_event: Option<&str>) -> Res
let mut rtmrs = [[0u8; 48]; 4];
// 각 RTMR (0-3)에 대해 재생
for idx in 0..4 {
let mut mr = [0u8; 48];

for event in eventlog.iter() {
// 1. 이벤트 무결성 검증
event
.validate()
```

```
.context("Failed to validate event digest")?;
// 2. 해당 IMR의 이벤트인지 확인
if event.imr == idx {
// 3. RTMR 확장: RTMR = SHA384(RTMR || event.digest)
let mut hasher = Sha384::new();
hasher.update(mr);
hasher.update(event.digest);
mr = hasher.finalize().into();
}
// 4. 특정 이벤트까지만 재생 (옵션)
if let Some(to_event) = to_event {
if event.event == to_event {
break;
}
}
}
rtmrs[idx as usize] = mr;
}

Ok(rtmrs)
}
```

5.3 특정 이벤트 찾기

```rust
// ra-tls/src/attestation.rs
impl<T> Attestation<T> {
fn find_event(&self, imr: u32, ad: &str) -> Result<EventLog> {
for event in &self.event_log {
// system-ready 이벤트 이후는 검색하지 않음
if event.imr == 3 && event.event == "system-ready" {
break;
}
if event.imr == imr && event.event == ad {
return Ok(event.clone());
}
}
Err(anyhow!("event {ad} not found"))
}

fn find_event_payload(&self, event: &str) -> Result<Vec<u8>> {
self.find_event(3, event).map(|event| event.event_payload)
}
}
```

6. 애플리케이션 정보 추출

6.1 AppInfo 구조체

```rust
// ra-tls/src/attestation.rs

/// Information about the app extracted from event log

#[derive(Debug, Clone, Serialize)]

pub struct AppInfo {

/// App ID

#[serde(with = "hex_bytes")]

pub app_id: Vec<u8>,

/// SHA256 of the app compose file

#[serde(with = "hex_bytes")]

pub compose_hash: Vec<u8>,

/// ID of the CVM instance

#[serde(with = "hex_bytes")]

pub instance_id: Vec<u8>,

/// ID of the device

#[serde(with = "hex_bytes")]

pub device_id: Vec<u8>,

/// TCB info

#[serde(with = "hex_bytes")]

pub mrtd: [u8; 48],
```

```rust
/// Runtime MR0-3
#[serde(with = "hex_bytes")]
pub  rtmr0: [u8; 48],
#[serde(with = "hex_bytes")]
pub  rtmr1: [u8; 48],
#[serde(with = "hex_bytes")]
pub  rtmr2: [u8; 48],
#[serde(with = "hex_bytes")]
pub  rtmr3: [u8; 48],
/// Measurement of everything except the app info
#[serde(with = "hex_bytes")]
pub  mr_system: [u8; 32],
/// Measurement of the entire vm execution environment
#[serde(with = "hex_bytes")]
pub  mr_aggregated: [u8; 32],
/// Measurement of the app image
#[serde(with = "hex_bytes")]
pub  os_image_hash: Vec<u8>,
/// Key provider info
#[serde(with = "hex_bytes")]
pub  key_provider_info: Vec<u8>,
}
```

6.2 AppInfo 추출 과정

```
// ra-tls/src/attestation.rs

impl<T> Attestation<T> {

/// Decode the app info from the event log

pub fn decode_app_info(&self, boottime_mr: bool) -> Result<AppInfo> {

// 1. Event Log 재생

let rtmrs = self

.replay_event_logs(boottime_mr.then_some("boot-mr-done"))

.context("Failed to replay event logs")?;

// 2. Quote 파싱

let quote = self.decode_quote()?;

// 3. Device ID 계산 (Quote 헤더의 user_data 해시)

let device_id = sha256(&[&quote.header.user_data]).to_vec();

// 4. TD Report 추출

let td_report = quote.report.as_td10().context("TDX report not found")?;

// 5. Key Provider 정보 추출

let key_provider_info = if boottime_mr {

vec![]

} else {

self.find_event_payload("key-provider").unwrap_or_default()

};

// 6. MR Key Provider 계산
```

```
let mr_key_provider = if key_provider_info.is_empty() {
[0u8; 32]
} else {
sha256(&[&key_provider_info])
};
// 7. MR System 계산
let mr_system = sha256(&[
&td_report.mr_td,
&rtmrs[0],
&rtmrs[1],
&rtmrs[2],
&mr_key_provider,
]);
// 8. MR Aggregated 계산
let mr_aggregated = {
use sha2::{Digest as _, Sha256};
let mut hasher = Sha256::new();
for d in [&td_report.mr_td, &rtmrs[0], &rtmrs[1], &rtmrs[2], &rtmrs[3]] {
hasher.update(d);
}
// 하위 호환성을 위해 모든 값이 0이 아닌 경우에만 포함
if td_report.mr_config_id != [0u8; 48]
|| td_report.mr_owner != [0u8; 48]
```

```
|| td_report.mr_owner_config != [0u8; 48]
{
hasher.update(td_report.mr_config_id);
hasher.update(td_report.mr_owner);
hasher.update(td_report.mr_owner_config);
}
hasher.finalize().into()
};
// 9. AppInfo 구성
Ok(AppInfo {
app_id: self.find_event_payload("app-id").unwrap_or_default(),
compose_hash: self.find_event_payload("compose-hash")?,
instance_id: self.find_event_payload("instance-id").unwrap_or_default(),
device_id,
mrtd: td_report.mr_td,
rtmr0: rtmrs[0],
rtmr1: rtmrs[1],
rtmr2: rtmrs[2],
rtmr3: rtmrs[3],
os_image_hash: self.find_event_payload("os-image-hash").unwrap_or_default(),
mr_system,
mr_aggregated,
key_provider_info,
```

```
})
}
}
```

---

7. TCB 검증

7.1 TCB 검증 함수

```rust
// ra-tls/src/attestation.rs
/// Validate the TCB attributes
pub fn validate_tcb(report: &VerifiedReport) -> Result<()> {
fn validate_td10(report: &TDReport10) -> Result<()> {
// 1. 디버그 모드 확인
let is_debug = report.td_attributes[0] & 0x01 != 0;
if is_debug {
bail!("Debug mode is not allowed");
}
// 2. MR Signer SEAM 검증
if report.mr_signer_seam != [0u8; 48] {
bail!("Invalid mr signer seam");
}
Ok(())
}
```

```
fn validate_td15(report: &TDReport15) -> Result<()> {
// 1. MR Service TD 검증
if report.mr_service_td != [0u8; 48] {
bail!("Invalid mr service td");
}
// 2. 기본 TD10 검증 수행
validate_td10(&report.base)
}
fn validate_sgx(report: &EnclaveReport) -> Result<()> {
// SGX 디버그 모드 확인
let is_debug = report.attributes[0] & 0x02 != 0;
if is_debug {
bail!("Debug mode is not allowed");
}
Ok(())
}
// 리포트 타입에 따른 검증 수행
match &report.report {
Report::TD15(report) => validate_td15(report),
Report::TD10(report) => validate_td10(report),
Report::SgxEnclave(report) => validate_sgx(report),
}
```

```
}
```

---

8. 인증서 통합

8.1 인증서에서 Attestation 추출

```
// ra-tls/src/attestation.rs

impl Attestation {

/// Extract attestation data from a certificate

pub fn from_cert(cert: &impl CertExt) -> Result<Option<Self>> {

Self::from_ext_getter(|oid| cert.get_extension_bytes(oid))

}



/// From an extension getter

pub fn from_ext_getter(

get_ext: impl Fn(&[u64]) -> Result<Option<Vec<u8>>>,

) -> Result<Option<Self>> {

// 1. Quote 확장 추출

let quote = match get_ext(oids::PHALA_RATLS_QUOTE)? {

Some(v) => v,

None => return Ok(None),

};

// 2. Event Log 확장 추출
```

```
let raw_event_log = get_ext(oids::PHALA_RATLS_EVENT_LOG)?.unwrap_or_default();
Self::new(quote, raw_event_log).map(Some)
}

/// Extract attestation from x509 certificate in PEM format
pub fn from_pem(cert: &[u8]) -> Result<Option<Self>> {
let (_, pem) =
x509_parser::pem::parse_x509_pem(cert).context("Failed to parse certificate")?;
let cert = pem.parse_x509().context("Failed to parse certificate")?;
Self::from_cert(&cert)
}
}
```

8.2 RA-TLS 인증서 생성

```
// ra-tls/src/cert.rs (추정)
pub fn generate_ratls_cert(key: &KeyPair) -> Result<String> {
// 1. 공개키 추출
let pubkey = key.public_key_der();
// 2. Report Data 생성
let report_data = QuoteContentType::RaTlsCert.to_report_data(&pubkey);
// 3. Quote 생성
let (_, quote) = get_quote(&report_data, None).context("Failed to get quote")?;
```

```
// 4. Event Log 읽기
let event_logs = read_event_logs().context("Failed to read event logs")?;
let event_log = serde_json::to_vec(&event_logs).context("Failed to serialize ev
// 5. 인증서에 Quote와 Event Log를 확장으로 포함
let cert = CertificateBuilder::new()
.subject("CN=dstack-ratls")
.public_key(key)
.extension(oids::PHALA_RATLS_QUOTE, quote)
.extension(oids::PHALA_RATLS_EVENT_LOG, event_log)
.build()?;
Ok(cert.to_pem())
}
```

요약

dstack의 TDX Quote 시스템은 다음과 같은 특징을 가집니다:

핵심 특징:

1. **하드웨어 기반 증명**: Intel TDX 하드웨어를 통한 신뢰할 수 있는 Quote 생성
2. **Event Log 재생**: RTMR 값 검증을 통한 실행 무결성 확인
3. **다양한 콘텐츠 타입**: KMS Root CA, RA-TLS 인증서, 앱 데이터 지원
4. **포괄적 검증**: Report Data, RTMR, TCB 속성 등 다층 검증

5. **인증서 통합**: X.509 인증서 확장을 통한 Quote 임베딩

검증 과정:

1. **Quote 파싱 → Report Data 검증 → Intel DCAP QVL 검증**
2. **Event Log 재생 → RTMR 비교 → TCB 속성 검증**
3. **애플리케이션 정보 추출 → 측정값 계산 → 신뢰성 확인**

이 시스템을 통해 외부 검증자는 dstack 애플리케이션이 정말로 TEE 환경에서 예상된 코드로 실행되고 있음을 수학적으로 증명할 수 있습니다.